Bloc 1 : COLLECTER, TRANSFORMER ET SECURISER LES DONNEES

Développer un pipeline ETL pour les données énergétiques

Osman SAID ALI
Ynov Toulouse
2025
M2 data scientist

Table des matières

I. Introduction . . . 3
II. Collecte de données (A1.1) . . . 4
II.1. Collecte du besoin (C1.1.1) . . . 4
Contexte . . . 4
Problématique . . . 4
Objectifs . . . 4
Définition de la stratégie de collecte . . . 4
II.2. Définition de la collecte (C1.1.2) . . . 5
II.3. Pipeline de collecte de données (C1.1.3) . . . 7
III. Stockage de données (A1.2) . . . 8
III.1. Stratégie de Stockage de données (C1.2.1) . . . 8
III.2. Construire une base de données (C1.2.2) . . . 9
IV. Structuration, transformation et enrichissement de données (A1.3) . . . 12
IV.1. Stratégie de traitement des données (C1.3.1) . . . 12
IV.2. Transformation des données (C1.3.2) . . . 13
IV.3. Mise en place d'un processus ETL (C1.3.3) . . . 15
V. Sécurisation des données (A1.4) . . . 16
V.1. Enjeux de sécurité et moyens déployés (C1.4.1) . . . 16
V.2. Schéma architecture, Stockage, flux (C1.4.2) . . . 18
VI. Conclusion . . . 19
VII. Annexe . . . 20

I. Introduction

Dans la course à l'intelligence artificielle, notamment au développement des grands modèles de langage, le rôle de l'ingénieur de données est parfois mis de côté. Pourtant, il est le pilier des stratégies d'IA, en permettant la mise en place d'infrastructures robustes capables de transformer des données brutes en informations exploitables, dont dépendent la qualité des modèles d'IA et des analyses.

L'ingénierie de données est un domaine qui vise à collecter, structurer et échantillonner les données de manière à choisir un traitement approprié. Ainsi, le but de l'ingénierie des données est de sélectionner, trier et organiser les données de sorte à valider leur condition et leur pertinence. De ce fait, la maîtrise de l'ingénierie de données est un facteur stratégique pour les entreprises pour assurer une croissance et une performance optimale.

Dans cette optique, ce projet se concentre sur le cas d'une entreprise fictive EnergIA, spécialisée dans la distribution d'énergie. Face à l'intégration accrue des énergies vertes qui provoque l'instabilité du réseau électrique, elle cherche à automatiser la collecte et l'analyse de ses données énergétiques pour fiabiliser les prévisions de production. La mission consiste à construire une architecture de données complète, depuis l'ingestion jusqu'à la sécurisation.

Ce rapport s'articule autour de quatre grandes parties : la collecte des données, leur stockage, leur traitement/transformation, et enfin leur sécurisation. Ce cadre vise à offrir une solution intégrée et moderne à la gestion des données dans le secteur énergétique.

II. Collecte de données (A1.1)

II.1. Collecte du besoin (C1.1.1)

Contexte

De nos jours, le nombre de producteurs d'énergies renouvelables qui se raccordent aux réseaux électriques augmente. Créant ainsi une variation importante de la production totale, non pas seulement en raison de leur multiplication, mais surtout à cause du caractère intermittent de ces sources d'énergie. Cette fluctuation implique une instabilité du réseau et exige une prévision des besoins.

Ainsi, l'entreprise EnergIA, spécialisée dans la distribution et le pilotage des réseaux électriques souhaite digitaliser, moderniser ses infrastructures et centraliser son système d'information. Pour bien piloter son réseau, elle doit mettre en place une architecture de traitement de données robuste. Celle-ci doit assurer des analyses fiables, une planification de la maintenance ainsi que des prévisions de qualité concernant les productions et la consommation. L'objectif est donc de développer un système de collecte, de traitement et de sécurisation de ces données.

Problématique

Actuellement, l'absence d'un système de traitement de données unifié limite la qualité des analyses. Ce qui engendre une réactivité lente face aux fluctuations des productions.

La problématique principale est donc de concevoir une architecture de collecte et de traitement de données capable d'intégrer des sources hétérogènes, tout en garantissant la qualité, la sécurité et la pertinence des données. Le tout pour permettre une gestion optimisée du réseau électrique et de préparer efficacement des modèles de prévision énergétique.

Objectifs

Les objectifs de ce projet sont les suivants :

- Automatiser la collecte via un pipeline de traitement de données.
- Migrer vers une infrastructure cloud sécurisée.
- Centraliser les données énergétiques et météorologiques dans une base de données.
- Assurer la disponibilité des données à 99% (24h/24, 7j/7).
- Intégrer des mesures de sécurité multicouches et le RGPD.

Définition de la stratégie de collecte

Type de donnée	Granularité temporelle	Source de donnée
Productions	1 heure	API interne SCADA
Consommation	15 minutes	API interne
Météorologique	1 heure	API (open Weather Map)

Tableau 1 : Stratégie de collecte des données

II.2. Définition de la collecte (C1.1.2)

La collecte de données est une méthode d'extraction de données via des API externes, des bases de données internes, des techniques de web crawling et de web scraping afin d'extraire les données ciblées.

Pour ce projet, trois types de sources de données ont été identifiés : les différentes productions, la consommation et la météo. La technique de collecte de ces données repose sur l'utilisation des API, pour automatiser, standardiser et fiabiliser l'extraction des données dans un système d'ingestion de données. Ce choix permet d'assurer une récupération planifiée, ou en temps quasi réel, tout en réduisant la dépendance à l'intervention humaine. Cette approche est illustrée par la figure 1.

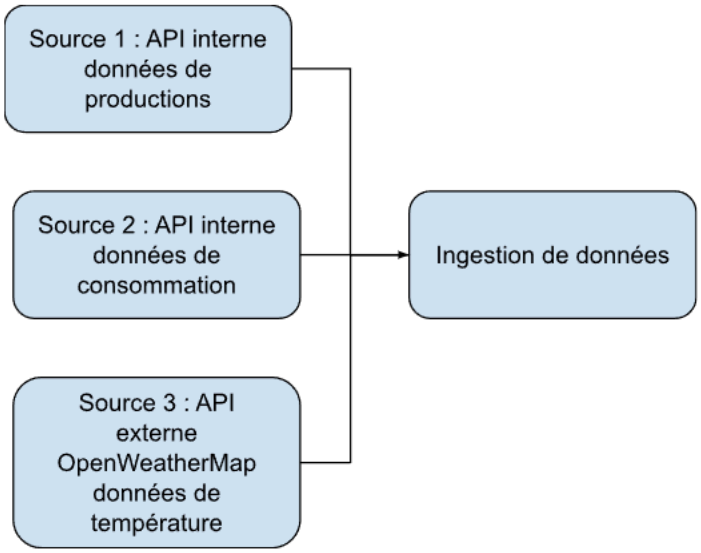

Figure 1 : Stratégie d'ingestion de données

<u>Techniques de collecte mises en œuvre</u>

Ces API sont bien documentées et sécurisées, permettant une intégration simple et fiable des données.

- Les données issues de sources internes (productions et consommations électriques) sont accessibles via des API avec un token d'accès de type Bearer.
- Ces API acceptent des paramètres spécifiques tels que le type de production ou de consommation, ainsi que les dates de début et de fin. Si aucun paramètre n'est fourni en entrée, elles renvoient les valeurs les plus récentes .
- Les données météorologiques sont collectées via l'API REST d'OpenWeatherMap, en utilisant des paramètres dynamiques (latitude, longitude, horodatage). Cette API impose également des quotas d'appel à respecter.

Les réponses retournées par ces APIs sont au format **JSON**, un format léger, standardisé et bien supporté dans les environnements de traitement de données. Cela facilite leur intégra-

tion dans des workflows automatisés en Python ou autres outils, via des bibliothèques telles que `Requests` , `Pandas` , `Polars` ou `PySpark` .

Chaque appel API est suivi d'une phase de structuration et de normalisation des données, permettant leur intégration homogène dans le système cible, comme illustré par la figure ci-dessus.

Pour garantir la fiabilité de la collecte, plusieurs mécanismes ont été mis en place :

- **Résilience** : jusqu'à 5 tentatives en cas d'échec, avec temporisation progressive entre les appels (gérée via le DAG (Directed Acyclic Graph)).
- **Monitoring** : journalisation (**logs**), avec alertes automatiques en cas d'erreurs ou de dépassement de quota.
- **Sécurité** : Authentification basique via HTTP pour obtenir un token valide, qui expire après chaque exécution. Et une authentification sécurisée de type Bearer est utilisée pour accéder aux données.

Exemple d'une requête API avec 5 tentatives

```python
@task
def extract_productions():
    """
    Extrait les différentes productions via l'API.

    Returns
    -------
    dict
        Données JSON de productions.
    """
    # Récupérer le token
    token = os.getenv("API_TOKEN")
    if not token:
       raise ValueError("Token API manquant")
    api_headers = {
            "Host": "digital.scada_service.energia.com",
            "Authorization": f"Bearer {token}"}

    endpoint = f"https://digital.scada_service.energia.com/open_api/type_production"
    api_response = requests.get(endpoint, headers=api_headers)
    if api_response.status_code == 200:
        return api_response.json()
    else:
        raise Exception(f"Échec de la récupération des données : {api_response.status_code}")
```

Ce bout de code montre l'extraction de données de productions automatisée issues d'une API interne (SCADA). Remarque importante, la gestion des erreurs (retries, timeout) se fait via Airflow (DAG).

II.3. Pipeline de collecte de données (C1.1.3)

La mise en place d'un pipeline de collecte de données joue un rôle clé pour assurer une extraction structurée, fiable et automatisée des données issues de trois types de sources principaux.

Un pipeline de collecte de données dans ce projet désigne donc l'ensemble des composants qui permettent :

- L'extraction des données depuis leurs sources API (internes ou externes).
- L'automatisation de cette extraction selon une fréquence définie.
- La planification des tâches à intervalles réguliers ou en quasi-temps réel.

Ce pipeline vise à :

- Respecter la fréquence d'actualisation propre à chaque source de données (tableau 1).
- Garantir la disponibilité, la traçabilité et la fiabilité des données collectées.
- Réduire les interventions manuelles grâce à une orchestration complète du processus.

Pour cela, il existe des outils d'orchestration qui permettent de réaliser un pipeline de collecte de données comme Apache Airflow, Apache Kafka, Prefect, Cron.

Outils	Planification	Monitoring	Cloud	Complexité	Avantages clés
Airflow	Avancée (DAGs)	UI web intégré	Oui	Moyenne	Visualisation, erreurs gérées
Kafka	Non	Très complet	Oui	Elevée	Le temps réel
Prefect	Avancée et flexible	UI web dédiée	Oui	Moyenne	Rapide à déployer, observabilité efficace
Cron	Basique	Aucun	Non	Très facile	Léger, facile à mettre en place mais peu robuste

Tableau 2 : Comparatif des orchestrateurs de workflow pour pipelines de données

Choix de l'outil d'orchestration

L'outil Apache Airflow a été retenu pour orchestrer le pipeline de collecte de données, en raison de ses avantages techniques adaptés aux exigences du projet :

- Modélisation claire des tâches via DAGs visuels.
- Interface web complète pour le monitoring et la supervision.
- Planification fine, conforme aux fréquences du projet.
- Tolérance aux erreurs avec reprise automatique et logs intégrés.
- Extensibilité et large communauté open source.

Exemple de dag

```python
from airflow import DAG
from airflow.operators.python import PythonOperator
```

```
from airflow.utils.task_group import TaskGroup
from airflow.operators.dummy_operator import DummyOperator
from datetime import timedelta
import pendulum
from extract_data import (extract_productions,
extract_energy_consumption, extract_weather_data)
# # Paramètres par défaut du DAG
default_args = {"owner": "airflow",
                "retries": 5,
                "retry_delay": timedelta(minutes=10),
  "start_date": pendulum.today('UTC').subtract(days=1)}
## Définition du DAG (nom, planning d'exécution toutes les heures)
with DAG(
      dag_id='api_dag',
       default_args=default_args,
       schedule="@hourly",
      catchup=False) as dag:
      # Tâche indiquant le début du pipeline
      start_pipeline = DummyOperator(
          task_id = 'Pipeline_collection_data_is_ready')
  # Parallélisation de tâches liées à l'ingestion des données
  with TaskGroup("Data_ingestion") as extract_group:
      # Extraction de données de consommation
      raw_consumption = extract_energy_consumption()
      # Extraction de données de production
      raw_productions = extract_productions()
      # Extraction de données de température
      raw_weather = extract_weather_data()
  start_pipeline >> extract_group
```

Ce choix permet de garantir une orchestration fiable, évolutive et industrialisable du pipeline de collecte de données. L'extraction de ces données (productions, consommation et température) sont parallélisées dans une seule tâche "TaskGroup" qui s'exècute toutes les heures. Un point très important concerne l'extraction des données de consommation : nous en extrayons 4 valeurs à chaque heure.

III. Stockage de données (A1.2)

III.1. Stratégie de Stockage de données (C1.2.1)

La stratégie de stockage constitue un élément clé du projet. Elle garantit la structuration, la disponibilité et l'accessibilité des données collectées. Ces données sont des données temporelles, qui seront utilisées pour des analyses avancées, de visualisation et de prévision énergétique.

Pour assurer une exploitation efficace et durable, il est nécessaire de concevoir un modèle de données flexible, capable d'évoluer dans un environnement en constante transformation, tout en préservant l'intégrité, la fiabilité et la cohérence des flux collectés. Pour cela, nous avons défini la structure de nos données, estimé la volumétrie, la disponibilité et ainsi que l'accessibilité.

Typologie des données à stocker

Type de données	Structure	Usage	Fréquence
Productions	Semi-structurée JSON à tabulaire	Analyse, prévision, visualisation	1 heure
Consommation énergétique	Semi-structurée JSON à tabulaire	Analyse, prévision, agrégation réseau	15 minutes
Météorologiques	Semi-structurée JSON à tabulaire	Corrélation, facteurs exogènes prévision	1 heure

Tableau 3 : Typologie des données

Volumétrie de données estimée

Le volume de données reste modéré mais en croissance constante. Sur une base annuelle :

- 96 enregistrements/jour/source(consommation).
- 24 enregistrements/jour/source(météo).
- 24 enregistrements/jour/source (productions), nous avons 4 sources de production (Solaire, Biomasse, Eolienne et Nucléaire).
- 6480 enregistrements/mois au total (les 6 sources).
- 77760 enregistrements/an (les 6 sources), soit environ 3.9 Mo/an.

La solution de stockage doit donc être scalable verticalement (performances) et horizontalement (modularité / archivage).

Disponibilité de données

- Base cloud sécurisée avec SLA de 99%.
- Sauvegardes automatiques et restaurations possibles à tout moment.
- Planification adaptée pour éviter les fenêtres de maintenance.

Accessibilité des données

- Contrôle d'accès par profils.
- Interfaces standardisées (clients SQL, outils de visualisation).
- Schémas organisés par type de donnée pour faciliter l'exploration.
- Indexation temporelle optimisée pour accélérer les requêtes.

III.2. Construire une base de données (C1.2.2)

La création d'une base de données nécessite une compréhension approfondie de la nature de données, des volumes et sa disponibilité. Il existe trois principaux types de structures données :

- Les données structurées : technologie SQL relationnelle adaptée nécessitant cohérence et modèles bien définis.
- Les données non structurées : technologie NoSQL utiles pour les données comme les images, vidéos ou autres, fortement volumiques ou distribuées.
- Les données semi-structurées : technologie SQL ou NoSQL, donc flexible et combine les deux.

Choix de technologie SQL ou NoSQL

À la lumière de la typologie précédente (tableau 3), le modèle relationnel SQL s'impose comme le choix le plus pertinent, grâce à :

- Une structuration claire en lignes et colonnes de nos données.
- Une intégrité renforcée via les types explicites et l'indexation temporelle.
- Une interopérabilité efficace avec les outils d'analyse et de visualisation.
- Une évolutivité maîtrisée avec des solutions cloud scalables (ex. AWS RDS, Google Cloud SQL).

Ce choix garantit un équilibre optimal entre performance, sécurité, maintenabilité et scalabilité, en adéquation avec le cycle de vie des données énergétiques du projet.

Modèle de données

Un modèle relationnel est utilisé pour organiser le stockage des données en base de données, structuré autour de trois tables principales :

- consommation (timestamp, Consommations)
- productions (timestamp, SOLAR, BIOMASS, WIND_ONSHORE, NUCLEAR)
- meteo (timestamp, température)

Modèle conceptuel de données

Pour une meilleure compréhension de la structure, un modèle conceptuel simplifié (MCD) est présenté ci-dessous. Il met en évidence les relations entre les jeux de données énergétiques et météorologiques, basées sur le temps comme clé de jointure commune.

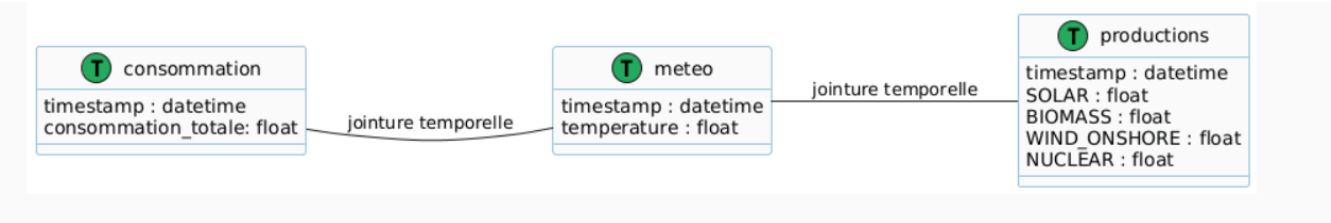

Figure 2 : MCD du projet

Choix des solutions de stockage

Les données utilisées dans ce projet sont de nature temporelle et organisées sous forme tabulaire (lignes/colonnes), avec une volumétrie limitée, ce qui oriente le choix vers l'utilisation d'un SGBD. Le système choisi est PostgreSQL, pour des raisons suivantes :

- Reconnu pour sa stabilité et sa maturité.
- Capacités d'indexation.
- Forte interopérabilité avec d'autres outils ou systèmes.
- Déploiement facile et scalable dans le cloud.

Ces atouts en font un système pertinent, tant pour les projets de petite que de grande envergure, ainsi, il permet de répondre aux besoins du projet en matière de performance, de

sécurité et de maintenabilité, tout en conservant une grande flexibilité pour accompagner de futures évolutions.

Exemple de création de table

```sql
CREATE TABLE IF NOT EXISTS consommation (
                timestamp TIMESTAMP PRIMARY KEY,
                Consommation FLOAT NOT NULL CHECK (Consommation > 0),
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );
CREATE TABLE IF NOT EXISTS meteo (
  timestamp TIMESTAMP PRIMARY KEY,
  temperature FLOAT
);
```

Chaque table est :

- Indexée sur le champ timestamp pour optimiser les jointures temporelles et les requêtes analytiques.
- Dotée d'une clé primaire temporelle unique, garantissant l'unicité des mesures.
- Conçue pour permettre une intégration fluide avec des outils de traitement en aval (Nettoyage, ETL, DataViz, etc.).

Paramétrage de la base

Plusieurs paramètres critiques ont été configurés dès la création de la base de données pour garantir une meilleure disponibilité et gestion des erreurs.

Paramètre	Valeur / Règle	Objectif
Indexation temporelle	Sur les colonnes `timestamp`	Accélération des requêtes
Sauvegardes automatiques	`@daily` 2h matin	Planification heure creuse
Intégrité de données Retry/ Timeout	5 tentatives/30 min	Gestion aux erreurs de sauvegarde
Rétention des données	30 jours	Maîtriser l'usage mémoire et permettre l'archivage
Monitoring Interface Airflow + logs	Web UI + fichiers JSON	Suivi, alertes et traçabilité
PostgreSQL (`pg_dump`)	`--format=plain` + `gzip -9`	Sauvegarde complète et compressée

Tableau 4 : Réglages critiques pour la fiabilité et l'intégrité des données

Remarque :

- Les solutions Big Data distribuées (ex. Hadoop, Spark, HDFS) n'ont pas été retenues, car les volumes des données à traiter restent modérés (moins de 80 000 enregistrements/an). Le choix d'une base SQL classique, optimisée et cloud native, permet d'assurer à la fois la scalabilité, la sécurité et la performance sans surdimensionner l'architecture.

IV. Structuration, transformation et enrichissement de données (A1.3)

IV.1. Stratégie de traitement des données (C1.3.1)

La stratégie de traitement des données est une phase essentielle de tout projet de données. Elle permet de définir les outils et technologies les plus adaptées au traitement de données collectées au préalable. Ainsi, les choix des outils et de technologies doivent tenir compte des spécificités de ce projet à savoir :

- La nature des données (séries temporelles).
- Les volumes de données.
- La nécessité d'un traitement régulier, automatisé, traçable et maintenable.

Outils et technologies de traitement de données

Il est important de faire la différence entre technologie et outil en ingénierie de données :

- Une technologie désigne une approche ou une méthode (exemple : ETL, traitement distribué),
- Un outil peut être un logiciel ou une implémentation d'une technologie (exemple : Airflow, Talend, Spark).

Cette compréhension de différence entre outil et technologie permettra de sélectionner les solutions les plus adaptées en fonction des contraintes du projet et des ressources disponibles.

Technologie	Avantages	Inconvénients
Traitement par lots	Simple, planifiable, peu coûteux en ressources	Pas adapté au temps réel, latence élevée
ELT (Extract Load Transform)	Centraliser le traitement dans le SGBD, réduit la duplication	Dépend fortement des performances du moteur de BDD. solution coûteuse.
ETL (Extract Transform Load)	Séparation claire des étapes, traitement optimisé avant stockage, meilleure gouvernance. Grande maturité.	Temps de traitement long, nécessite stockage intermédiaire
Orchestration de traitement	Automatisation, traçabilité, gestion d'erreurs, supervision visuelle	Configuration parfois complexe, nécessite maintenance des DAGs
Traitement distribué (Spark)	Scalable pour gros volumes, haute performance parallèle	Surdimensionné pour ce projet (≈ 77760 lignes/an, < 3 Mo)

Tableau 5 : Comparaison des technologies de traitement de données

Pourquoi Spark n'est pas retenu :

Le projet traite environ 77 760 enregistrements par an, représentant moins de 3.9 Mo de données par an. Spark ou Hadoop, conçus pour des données volumineuses (de l'ordre du Go au To), seraient donc excessifs ici et alourdiraient inutilement l'architecture.

Outils de traitement de données

Outil	Avantages	Limites
DBT (Data Build Tool)	Versioning SQL, documentation intégrée	Dépend de SQL
Airflow	Orchestration puissante, DAGs visuels, intégration possible avec dbt ou autres outils	Pas conçu pour faire les transformations directement
Apache Spark	Très puissant pour gros volumes, traitement distribué	Nécessite cluster, maintenance complexe
Talend	Interface graphique, connecteurs variés	Lourd sans équipe dédiée, coûteux à long terme
Prefect	Orchestration moderne, logs détaillés, cloud-native	Moins mature qu'Airflow dans les environnements critiques

Tableau 6 : Comparaison des outils de traitement de données

Choix de technologie et d'outil :

- **Technologie :** ETL (Extract, Transform, Load), car elle sépare clairement les étapes et permet une gouvernance des flux.
- **Outil** : Apache Airflow (orchestration) + Python (traitement) + Pandas (transformation des données).

Cette combinaison permet :

- Une automatisation planifiée et centralisée des traitements.
- Une traçabilité complète via les logs Airflow.
- Une souplesse de développement avec Python pour gérer le format JSON, les conversions de date, la détection d'anomalies, etc.

IV.2. Transformation des données (C1.3.2)

La transformation des données a pour but de préparer les données brutes issues de la collecte afin de les rendre cohérentes, nettoyées, normalisées et exploitables pour les étapes suivantes : analyse, visualisation ou modélisation (prévision énergétique notamment).

Les transformations appliquées s'effectuent en Python (Pandas), au sein de pipelines orchestrés avec Apache Airflow, et comprennent :

- Conversion des formats :
  - Données issues des API au format JSON transformées en structure tabulaire normalisée.

- Normalisation des unités :
  - Conversion cohérente des mesures (ex : K à °C) pour éviter les erreurs dans les visualisations et analyses.
- Suppression des doublons et remplissage des valeurs manquantes par interpolation linéaire.

Pipeline de transformation

Le pipeline se structure en trois grands blocs :

- Déclenchement : Pipeline_collection_data_is_ready.
- Ingestion de données : Data_ingestion (météo, productions et consommation).
- Transformation de données : Transform_datasets.

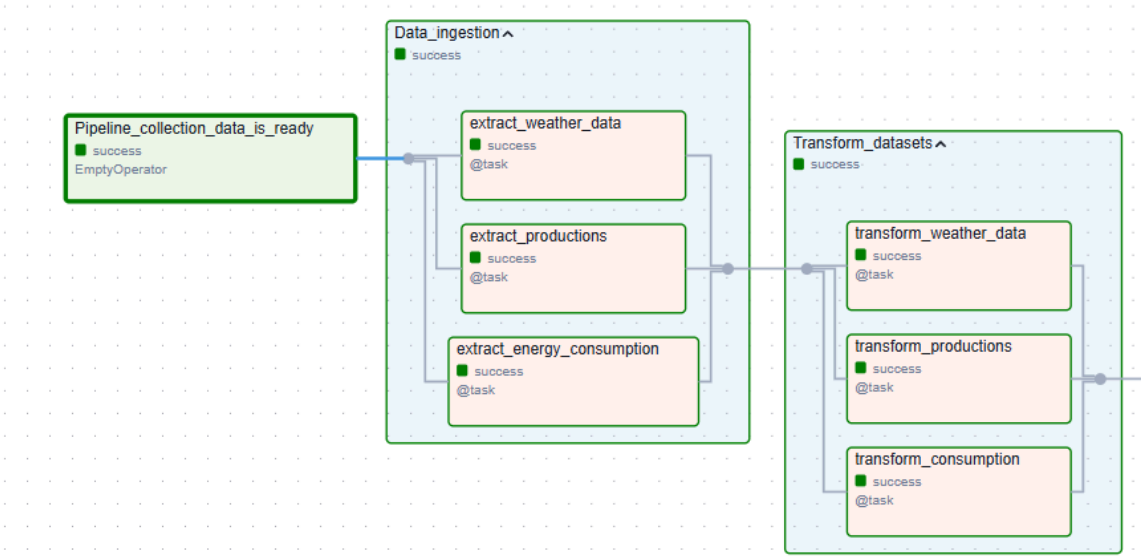

Figure 3 : Un pipeline d'extraction et de transformation

Un exemple de code de transformation est disponible en annexe.

Choix des outils et langage

- Langage : Python, pour sa souplesse et sa lisibilité.
- Librairie : Pandas, idéale pour le traitement des séries temporelles sur des volumes de données modérés. Pour des volumes de données plus important, il serait intéressant d'opter pour la librairie Polars ou Pyspark.
- Orchestrateur : Apache Airflow, pour automatiser et planifier les DAGs.

Pourquoi ne pas utiliser dbt ou Talend ?

- Talend : adapté à des équipes avec besoins d'interfaces visuelles, mais trop lourd pour notre volumétrie modérée.
- DBT : puissant mais limité au SQL. Certaines transformations ici par exemple horodatage nécessitent une logique impérative plus fluide en Python.

IV.3. Mise en place d'un processus ETL (C1.3.3)

Le processus ETL (Extract, Transform, Load) mise en place pour le projet est le pilier du pipeline de données. Il garantit un traitement structuré, automatisé et fiabilisé des données énergétiques et météorologiques, de leur extraction à leur mise à disposition en base PostgreSQL.

Architecture du pipeline ETL

Le pipeline suit la structure classique de l'ETL, avec les particularités suivantes :

Extract

- Données extraites via APIs REST sécurisées :
  - Interne : productions et consommation.
  - Externe : données météorologiques (OpenWeatherMap).
- Formats de sortie : toutes les APIs retournent des JSON.
- Planification :
  - Par heure.
- Résilience : 5 tentatives par appel API, avec journalisation automatique des erreurs (via logging Airflow).

Transform

- Conversion des formats (JSON à DataFrame tabulaire).
- Normalisation des unités (energies en MW, température en °C).
- Suppression des doublons et interpolation valeurs manquantes.
- Réalisation via Python/Pandas, orchestrée dans Apache Airflow.

Load

- Insertion des jeux de données transformés dans PostgreSQL, dans trois tables :
  - consommation (timestamp, consommations)
  - productions (timestamp, SOLAR, BIOMASS, WIND_ONSHORE, NUCLEAR).
  - meteo (timestamp, temperature).
- Vérification d'intégrité : gestion des conflits (doublons temporels).
- Indexation temporelle pour optimiser les performances en lecture.

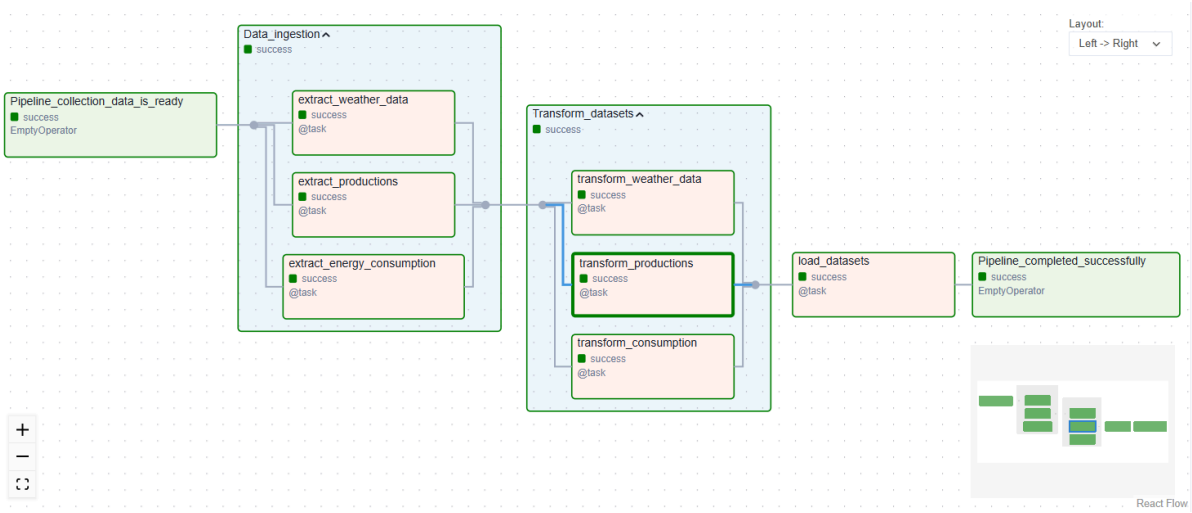

Figure 4 : Pipeline ETL (Extract, Transform and Load) du projet

Le pipeline ETL présente plusieurs bénéfices majeurs dans le contexte du projet :

- Flexibilité : traitement en Python pour gérer des cas spécifiques (ex. parsing JSON, vérifications métiers).
- Traçabilité complète : logs, DAGs visualisables, relances automatiques ;
- Conformité RGPD : la centralisation des données dans un entrepôt permet de contrôler les accès et d'assurer l'effacement ou la pseudonymisation des données sensibles si nécessaire avant le stockage.

Le code complet du pipeline est accessible sur github :

https://github.com/OsmanOSA/projet_data_engineer/blob/main/dags/pipelines_etls.py

V. Sécurisation des données (A1.4)

V.1. Enjeux de sécurité et moyens déployés (C1.4.1)

Avant toute mise en production du pipeline de données développé, il est nécessaire de mettre en place une politique de sécurisation robuste afin de garantir la confidentialité, l'intégrité et la disponibilité des données. Cette politique repose sur une analyse des risques, une gestion des accès, une traçabilité des actions et le respect des exigences réglementaires, notamment celles du RGPD.

Analyse des risques

Plusieurs risques ont été identifiés :

Type de risque	Impact potentiel	Mesures de sécurité déployées
Perte de données	Indisponibilité des données, perte d'historique critique, pas d'analyse et de prévision	Sauvegardes automatiques, versioning, redondance des volumes
Accès non autorisé	Fuite d'informations, manipulation ou suppression malveillante	Contrôle d'accès par rôle et masquage des identifiants via un fichier `.env` (voir code `SQL` en exemple)
Altération des données	Résultats d'analyse faussés, décisions erronées, perte de fiabilité du système	Limitation des droits d'écriture, logs Airflow
Non-conformité réglementaire	Sanctions juridiques, perte de confiance, violation du RGPD	Registre de traitement, traçabilité des accès, durée de rétention définie

Tableau 7 : Risques identifiés, avec leur impact et leur mesure d'atténuation

Exemple de Contrôle d'accès

L'accès à la base de données PostgreSQL est limité à certains profils, à travers une gestion des droits d'accès basée sur les rôles (Role-Based Access Control). Chaque rôle dispose de permissions adaptées à sa fonction :

```sql
CREATE ROLE lecteur_donnees;
GRANT CONNECT ON DATABASE airflow TO lecteur_donnees;
GRANT USAGE ON SCHEMA public TO lecteur_donnees;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO lecteur_donnees;

CREATE ROLE analyste_donnees;
GRANT lecteur_donnees TO analyste_donnees;
GRANT CREATE ON SCHEMA public TO analyste_donnees;
GRANT TEMPORARY ON DATABASE airflow TO analyste_donnees;

CREATE ROLE admin_donnees;
GRANT ALL PRIVILEGES ON DATABASE airflow TO admin_donnees;
```

Un fichier .env sécurisé est utilisé pour stocker les informations sensibles (clés API, identifiants), pour éviter toute exposition directe dans le code.

Appels API.

Tous les appels API sont effectués via le protocole HTTPS, garantissant le chiffrement des données en transit. L'authentification repose sur des tokens Bearer, renouvelés de manière sécurisée.

```
api_headers = {"Host": "openweathermap.com",
            "Authorization": f"Bearer {token}",
            "Accept": "application/json"
        }
https://api.openweathermap.org/data/2.5/weather?
```

Par ailleurs, les accès à la base PostgreSQL sont restreints à un réseau privé, avec filtrage par adresses IP autorisées. Un tunnel SSH est envisagé pour renforcer la sécurité lors des accès distants.

Conformité réglementaire (RGPD)

Même si les données manipulées ne concernent pas des individus identifiables, la politique de conformité au RGPD est appliquée par défaut :

- Rétention des données limitée à 30 jours (configurable).
- Registre de traitement tenu à jour.
- Accès restreint aux seules personnes autorisées.

Élément sécurisé	Technologie ou pratique mise en œuvre
Appels API	HTTPS + authentification par token (Bearer)
Orchestration	Airflow avec logs horodatés et stockage sécurisé
Base de données	PostgreSQL + accès restreint IP
Droits d'accès	Gestion RBAC (Role-Based Access Control)
Sauvegardes	Backups quotidiens automatisés avec Airflow (DAG)
Logs, Centralisés	Horodatés, persistants

Tableau 8 : Politique de sécurité

V.2. Schéma architecture, Stockage, flux (C1.4.2)

La sécurisation des données ne se limite pas aux droits d'accès ou aux sauvegardes : elle doit être intégrée dès la conception de l'architecture globale. Dans le cadre du projet EnergIA, une architecture sécurisée, robuste et évolutive a été mise en place pour garantir la protection des données en transit et au repos, tout en assurant leur disponibilité.

Architecture du projet

L'architecture repose sur une approche modulaire articulée autour de plusieurs couches de sécurité :

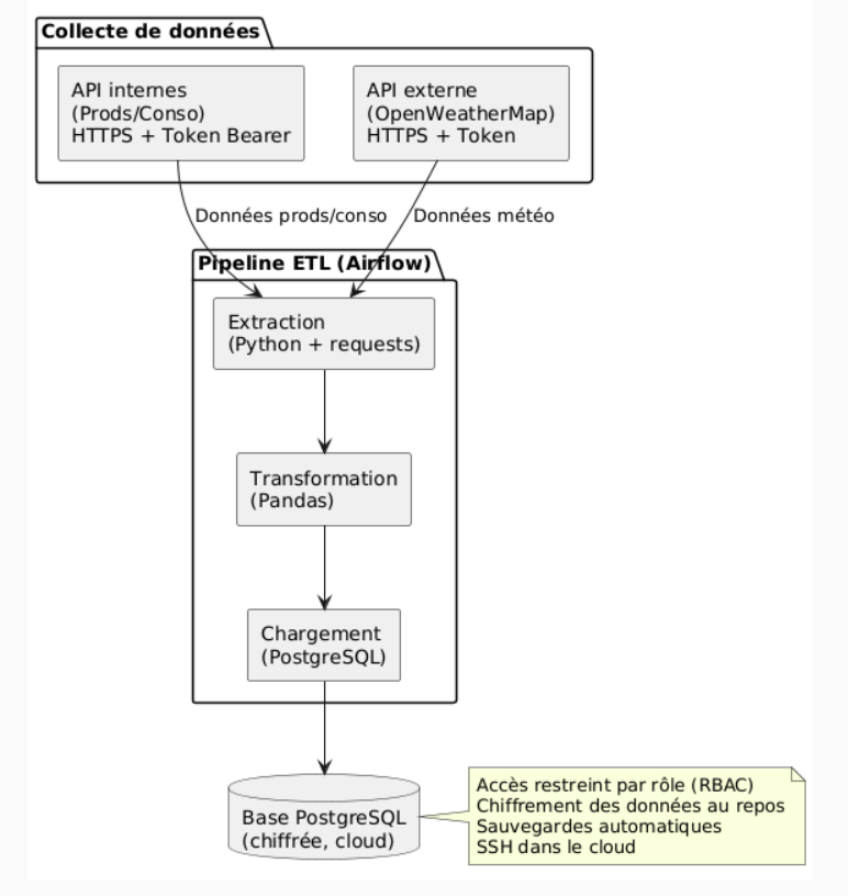

Figure 6 : Architecture sécurisée du pipeline de données

VI. Conclusion

Arrivés au terme de notre développement, nous pouvons conclure que ce projet a permis de mettre en place un pipeline de collecte, de traitement et de sécurisation de données robuste. Tout d'abord, nous avons collecté et analysé minutieusement les besoins, définissant ainsi les objectifs et la stratégie de collecte pour la suite de notre travail. Ensuite, nous avons mis en place une technique de collecte de données exploitant les API internes et externes, aboutissant à un pipeline de collecte de données orchestré par Airflow.

Avant le traitement, nous avons défini une stratégie de stockage garantissant l'intégrité, la disponibilité et l'accessibilité des données. Cela nous a conduit à choisir PostgreSQL, la base de données la plus adaptée pour ce projet.

Nous avons ensuite sélectionné la technologie ETL (Extract, Transform, Load), avec Python pour le traitement et Airflow pour l'orchestration. Les données ont été transformées avec Pandas, particulièrement efficace pour des séries temporelles de volume modéré.

Une fois nos données transformées, elles sont stockées dans la base de données PostgreSQL déjà préparée en amont.

Enfin, après avoir développé un pipeline ETL, nous avons mis en place des mesures de sécurité multicouche pour protéger nos données et assurer la conformité au RGPD. Nous avons limité les accès aux données selon les rôles.

En conclusion, ce projet souligne l'importance de l'ingénierie des données quel que soit le secteur (médical, énergie, finance, etc.).

VII. Annexe

```python
import os
import requests
import pandas as pd
import time
import pendulum
import logging
from airflow.decorators import task

@task
def transform_consumption(raw_consumptions: dict)-> pd.DataFrame:
    """
    Transforme les données de consommation extraites en DataFrame pandas.
    Parameters
    ----------
    raw_consumptions : dict
        Données brutes de consommation
    Returns
    ------
    pd.DataFrame
        DataFrame avec colonnes 'timestamp' (format string YYYY-MM-DD HH:MM)
        et 'Consommations' (valeurs en MW)
    """
    try:
        if not raw_consumptions or 'short_term' not in raw_consumptions:
            raise ValueError("Données de consommation invalides")
        values = raw_consumptions['short_term'][0]['values']
        if not values:
            raise ValueError("Aucune valeur de consommation trouvée")
        df_list = [
            {"timestamp": entry.get("start_date"),
                "Consommations": entry.get("value")}
            for entry in values if entry.get("start_date") and entry.get("value") is not None]
        df = pd.DataFrame(df_list)
        # Conversion de la colonne date en datetime
        df['timestamp'] = pd.to_datetime(
            df["timestamp"],
            utc=True).dt.strftime("%Y-%m-%d %H:%M")
        # Suppression des doublons
        df = df.drop_duplicates().set_index('timestamp')
        df.index = pd.to_datetime(df.index)
        # Remplir les valeurs manquantes
        df = df.fillna(df.interpolate('linear')).reset_index()
        logging.info(f"Transformation consommation terminée: {len(df)} enregistrements")
        return df
    except Exception as e:
        logging.error(f"Erreur de transformation: {str(e)}")
        raise AirflowException(f"Échec de transformation: {str(e)}")
```